

COMPUTER VISION-BASED SMART TRAFFIC CONGESTION ANALYTICS WITH ADAPTIVE SIGNAL OPTIMIZATION

A camera, a GPU and three microcontrollers that watch real traffic, measure it in km/h, and change the signals to suit it

Zeyad Khairy · [Member 2] · [Member 3] · [Member 4] · [Member 5]
School of Computing · Asia Pacific University of Technology & Innovation
Group Design Project · Final Submission · 14 weeks

PROJECT TEAM

1 Zeyad Khairy

Computer Vision & Detection

Detection and tracking, km/h speed from road-plane calibration, the congestion index.

2 [Member 2]

Dashboard & System Integration

The Streamlit control centre, telemetry logging, the database and the link to the CV node.

3 [Member 3]

Hardware & Control

The ESP32 hub, the Arduino signal heads, the servo lane changer, the LED strip and the LCDs.

4 [Member 4]

Emergency Vehicle Priority

Flashing-beacon detection, livery recognition, and the priority response that clears a lane.

5 [Member 5]

Testing, Calibration & Documentation

Speed calibration, the automated test suites, the measured metrics and the report.

27.1
fps end to end

YOLO11-m @ 1280 px, FP16, RTX 4050

±5%
speed accuracy

calibrated — exact vs. ground truth

0
false alarms at night

down from 29% of all traffic

831/833
automated checks pass

7 suites, reproducible

21,526
vehicles logged

across 69 recorded sessions

Introduction

why this exists

Urban signals overwhelmingly run on **fixed timers**. They hold a green over an empty approach and a red over a queue, because they cannot see the road they govern. Where sensors do exist they are usually inductive loops buried in the tarmac — expensive to install, blind to what kind of vehicle passed, and unable to say how fast anything was going.

This project replaces that with a **camera and a graphics card**. One video feed is turned into engineering measurements — a vehicle count, a speed in km/h, a congestion index between 0 and 1 — and those measurements drive real signal hardware, a movable lane divider and an emergency-vehicle response.

Three things happen to every frame:

- **SEE** — YOLO11-m detects vehicles at 1280 px and BoT-SORT keeps an identity on each one across frames.
- **MEASURE** — pixels are converted to metres on the road plane, giving a true km/h, and a weighted density and speed deficit are combined into one congestion index.
- **ACT** — a demand-responsive signal, a servo lane divider and a “make space” LED strip are driven over one USB cable, and every second is written to a spreadsheet.

The system runs on whatever it is pointed at — the camera on the desk, an uploaded video, or a public roadside stream URL — with the **same pipeline** and no restart.

Problem & Motivation

the three gaps

A camera-based traffic system is easy to demonstrate badly. Three specific failures sit between a demo and a measurement, and closing them is what this project is actually about.

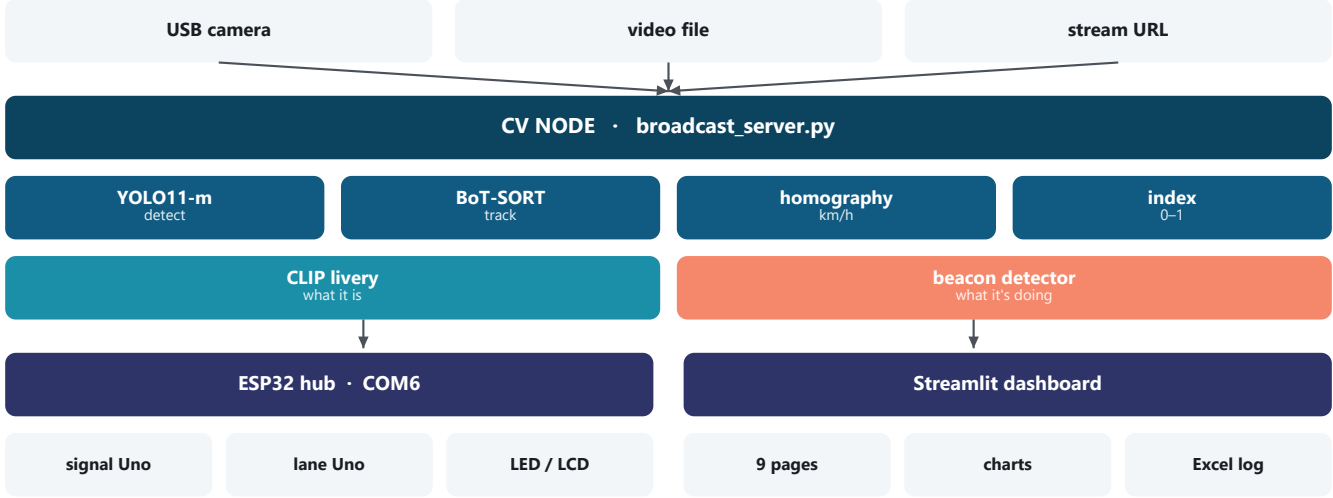
- **Pixels are not metres**. A distant car crossing 3 px per frame and a near car crossing 30 px per frame can be travelling at exactly the same speed. Any system reporting “pixels per second” has not measured a speed at all.
- **The computer’s clock is not the road’s clock**. Replay footage on a slow machine and the traffic appears slow; replay it on a fast one and it appears to speed. A reported speed must not be a property of the hardware watching it.
- **Looking like an ambulance is not being on a call**. An ambulance parked outside a hospital is still an ambulance. Appearance alone flagged **88 of 299 vehicles — 29% of all traffic** — on our night footage.

Objective. To build a working smart-traffic analytics prototype that measures congestion from live video and adapts signal timing to it, with every claim backed by a stated method rather than an impression.

System Architecture

two programs, one pipeline

The system is **two programs, deliberately separated**, because they need different things. The CV node has to be where the camera and the GPU are. The dashboard only needs a browser — so it can run on a laptop in another room or another building.



Every frame follows one path. The source is chosen while running, so the detector, the speed estimator, the congestion index, the signal and both workbooks are identical whatever is being watched.

Only the node holds the USB port. Windows will not let two programs open the same COM port, so a dashboard button does not talk to the Arduino — it asks the node to. That single rule is what lets the dashboard live on a different computer.

Hardware & Control

one USB cable

One USB cable leaves the PC and everything hangs off it. The **ESP32** is the hub; it relays the seven-field signal line down two UARTs to a pair of Arduino Unos that own the physical outputs.

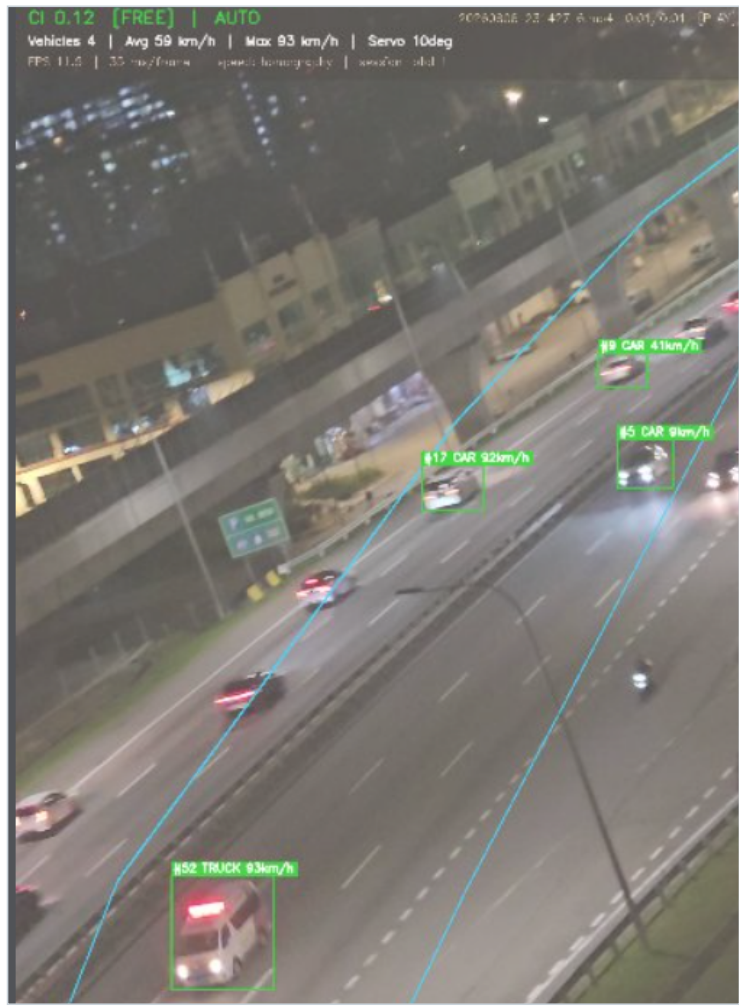
Board	Link	Drives
ESP32 hub	USB - COM6	WS2815 strip (60 px), 2× 1°C LCD
Signal Uno	GPIO33 → D7	Two ramp signal heads + status LCD
Lane Uno	GPIO25/26	Servo divider, 18-px strip, LCD

Grounds are tied together; the Unos need power, not a data cable.

- **Servo lane divider** — three positions (50° / 90° / 130°) reallocating lanes toward the busier direction.
- **Emergency LED strip on pin 6** — the “make space” output, separate from the signal heads.
- **Delivery is acknowledged**. Every command’s reply says whether it reached the wire, so an unplugged board is visible instead of a button that pretends it worked.

The System Running

night is the hard case



Night footage, live. The overlay reports the congestion index, the level, the signal phase and the servo angle; each box carries a tracking ID, a class and a speed in km/h. The cyan polygon is the region of interest — only vehicles inside it are counted, so the opposite carriageway does not inflate the measurement.

Measuring Speed in km/h

the load-bearing measurement

Speed is the measurement the whole system rests on, so it is taken on the **road plane in metres**, never in pixels. Two calibration modes are offered.

- **Automatic (±25%, works immediately)**. A car is about 1.5 m tall and a bus about 3.2 m, so the box height in pixels gives the metres-per-pixel scale at *that exact spot*. Every vehicle carries its own scale, so near and far are both handled.
- **Calibrated (±5%, once per camera position)**. Four corners of a road rectangle are clicked and its real size typed in — a lane is about 3.5 m. A homography then maps the whole road plane properly.

Three rules turn a position into a speed:

- The **bottom-centre** of the box is used — the only part of a vehicle actually touching the road.
- Displacement is measured over **0.8 s**, not between two frames, so box jitter is not reported as motion.
- Anything over **200 km/h** is discarded as a tracker identity swap rather than a vehicle.

Validation test	Expected	Measured
Homography mode	18 km/h	18.00
Homography mode	36 km/h	36.00
Homography mode	72 km/h	72.00
Height-based (auto)	18 km/h	18.00
Parked vehicle	0 km/h	0.12

Constructed ground truth: the vehicle is synthetic, so its speed is known rather than estimated.

A video gets its own clock. A vehicle scripted at 16.2 km/h was played at real time, 2×, half speed and unthrottled. All four runs measured **16.2 km/h**, agreeing to 0.00 — so the reported speed is a property of the traffic, not of the computer.

The Congestion Index

0 → 1, and honest at both ends

One number between 0 and 1, from two ingredients: **how slow everyone is (60%)** and **how full the road is (40%)**. Occupancy is weighted, because a bus does not take the same room as a motorbike.

Motorcycle	Car	Bus	Truck
0.5	1.0	3.0	3.0

Passenger-car-equivalent weights used for the occupancy term.

Two guard rules stop it saying something stupid:

- **An empty road is never congested**. The first version remembered the last speed it saw, so an empty road could still report HEAVY. Zero vehicles now means zero congestion, full stop.
- **One slow car is not a jam** — it is one slow driver. The speed term only counts once roughly 3+ vehicles are present to be slow together.
- **Hysteresis on the label**, so FREE / MODERATE / HEAVY does not flicker on the boundary — the LEDs and the servo would physically chatter.

Adaptive Signal Control

demand-responsive

The signal is **demand-responsive**. In the default merge layout the heads sit on the side roads feeding two carriageways, so they hold merging traffic back exactly when the main road is busiest.

Congestion on main road	Side-road signal
HEAVY	RED — close the merge
MODERATE	GREEN — there is room
FREE (empty)	GREEN
any change	YELLOW for 5 s first



- **Transitions are committed**. Once yellow starts it runs the full five seconds and always lands on the phase it was aiming at — real signals never abort an amber.
- **Minimum dwell of 3 s** on green and red, or traffic flipping during the yellow could produce a green lasting one frame.

Driven against a **synthetic clock** rather than by sleeping, so the interval is asserted exactly. With congestion flipping every 0.5 s for two minutes, no phase was ever momentary and the R→Y→G→Y→R ordering never broke.

An emergency vehicle does not seize the signal. Emergency traffic may proceed through any indication, so taking the light over would only disrupt everyone else. A dedicated LED strip tells nearby drivers to leave space and the LCD names the vehicle, while the signal carries on serving normal traffic.

Emergency Vehicle Priority

two cues, one decision

YOLO has never been taught the word *ambulance* — it reports “truck”. Two recognisers fill the gap, and keeping them **separate is the whole design**, because they answer different questions.

Recogniser	Asks	Role
CLIP V11-B/16	What kind of vehicle is that?	Names it
Beacon detector	Is it on a call right now?	Triggers priority

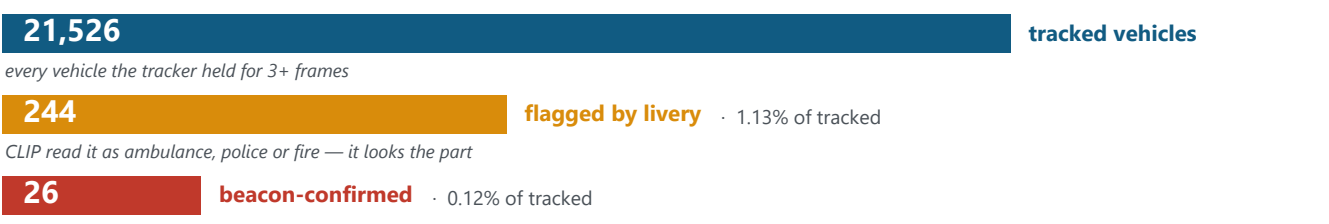


Recognised on unseen internet footage the system was never tuned for. CLIP is **zero-shot** — nothing was trained on this project’s cameras, which is why it travels between countries.

The beacon detector is not an AI at all — it is arithmetic about light and time. It looks only at the roof band (stretched 18% above it, since a lamp glows past its housing, and safely clear of brake lights), demands a properly saturated red or blue above 165/255, then starts a stopwatch.

- At least **3 flashes** inside a 3.4 s window.
- A rate of **1–6 Hz** — real beacons are regulated by law.
- It must go properly **out** between flashes, not merely dim.

Switch versus slide. A lamp is either on or off and is almost never caught halfway; a red car passing under a street light spends most of its time halfway. On the night footage the real beacon scored **0.00** and every false positive **0.18–0.43** — complete separation.



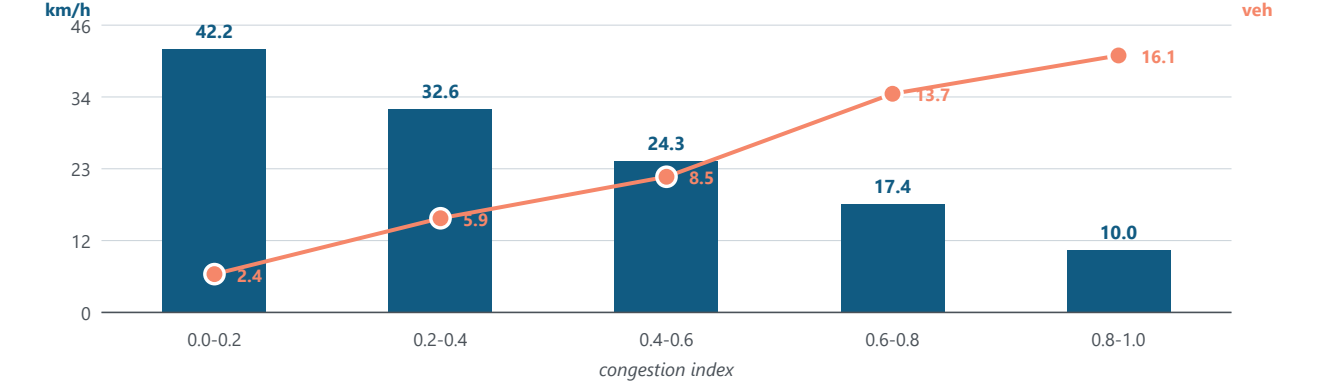
The two cues in the session archive. 244 vehicles looked the part; 26 were actually working. Recognising the difference is what stops a parked ambulance holding a junction.

Livery alone was tried as the trigger and rejected. Making the beacon the trigger took false alarms on the night footage from **88 of 299 vehicles (29%) to zero**, while still catching every planted emergency vehicle.

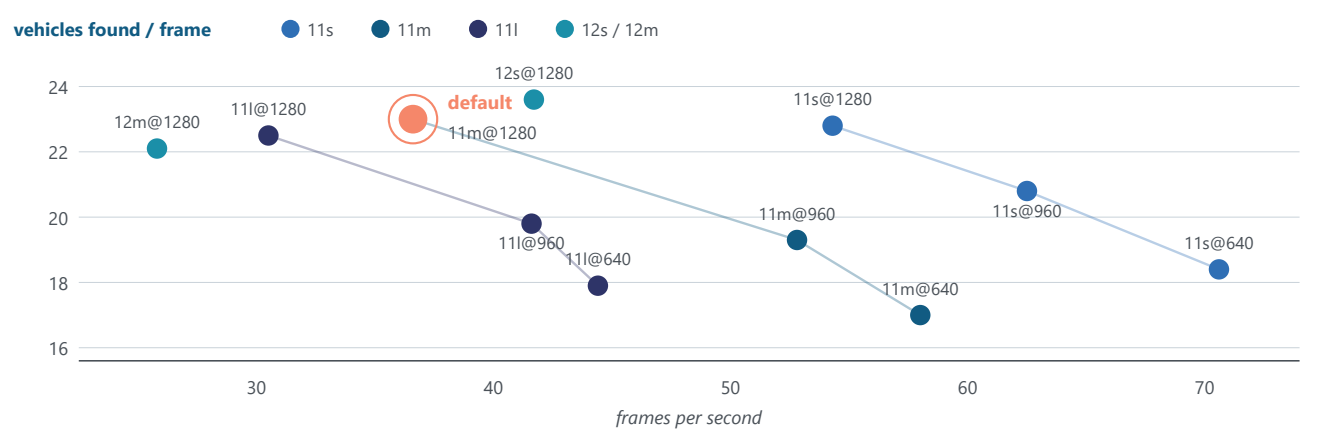
Results

measured, not estimated

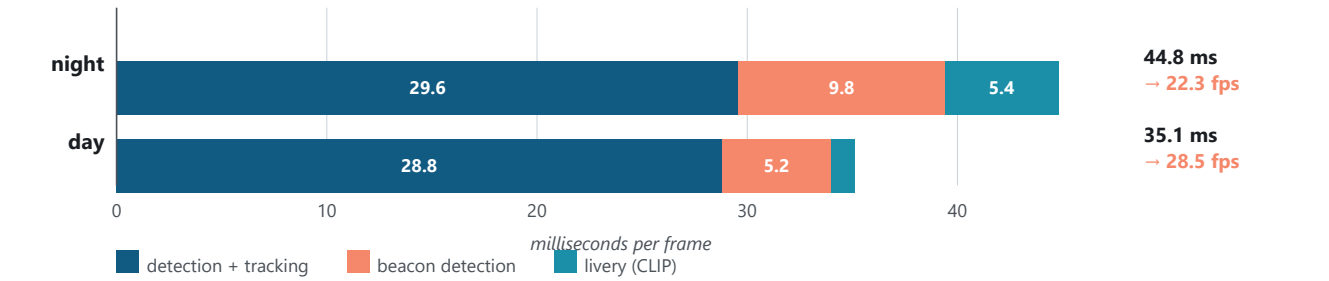
Congestion costs speed, and the archive shows it. 12,919 one-second samples across 66 recorded sessions, binned by congestion index. Mean speed falls monotonically as the index rises, while the vehicle count more than sixfolds.



Bars: mean speed in km/h. Line: mean vehicles present. The index was built to be read this way — it is not a proxy for the vehicle count alone.



Resolution buys detections; model size does not. Every family gains ~5 vehicles/frame from 640 → 1280 px, but at fixed resolution the small model matches the large one — yolo11-t4 @ 1280 finds fewer than yolo11-m for 20% more time. YOLO12 is the wrong family here: 12m is the slowest point on the chart and finds fewer.



Detection is two-thirds of the budget, so it is the only stage worth optimising. Both recognisers cost more at night — CLIP because dark crops fail the readability gate, the beacon detector because at night there are beacons to measure.



21,526 tracked vehicles by class, across the whole archive.

Control Centre & Data

9 pages · 12,919 samples

The dashboard is a nine-page Streamlit application that never touches the camera or the serial port itself — it reads the node’s telemetry over HTTP and asks the node to act. That is what lets it run on a different machine from the one doing the seeing.

Page	Answers
Operations	health, count, density, phase
Live Camera	the annotated feed + detections table
Traffic Analytics	trends across every session
Emergency Control	priority state, manual override
Hardware Monitor	board, LEDs, servo, LCD
Logs · Settings · About	diagnostics and configuration

Two diaries, because they answer two questions. traffic_history holds the current run and is what a demo shows; traffic_all_sessions is appended to and never reset. CSVs are written **every second** and are always complete; the Excel workbooks are built on request, so a download is never stale.

- Sheets: **Live Status, Traffic Log, Vehicle Speeds, Session Summary** — plus All Sessions, All Records and Lifetime Summary in the archive workbook.
- **One row per vehicle** that passed through, with its class, dwell time, distance, and average and top speed.
- Every chart carries a *how to read this note*, and a panel turns the numbers into plain sentences — the busiest hour, whether jams are constant or occasional, what congestion costs in speed.

Frozen numbers are reported as frozen. When a feed ends the dashboard says so rather than showing the last values as though they were live — the same principle as the acknowledged hardware command.

Validation & Honesty

833 checks · 7 suites

Suite	Checks	Covers
test_traffic_system.py	217	vision pipeline
test_server.py	165	hardware, HTTP, relay
test_video_source.py	113	sources, media clock
test_dashboard_interactive.py	109	widgets, exports
test_analytics.py	90	analytics maths
test_dashboard.py	83	all 9 pages render
test_traffic_light.py	56	signal timing
Total	833	

831 of 833 pass in one run, and 833 is not reachable in a single run. The two that do not are the pair that POST to the CV node; they pass when it is up, but four Live Camera checks written against the simulated feed then fail. No single run can satisfy both states. **831 is the honest, reproducible number**.

What this project does *not* claim:

- **No mAP is quoted**. There are no hand-drawn boxes for this footage, so there is nothing to score against. benchmark.py refuses to print an accuracy figure rather than borrow the model’s published COCO score.
- **6.6% is not an accuracy**. On an 11:43 unseen compilation the recogniser singled out 166 of 2,511 tracked vehicles. That is a selectivity figure; confirming a false-positive rate would mean labelling all 166 by hand.
- **A known open question, deliberately not acted on**. 11s@1280 reaches 99% of 11m’s yield at 48% more throughput — but vehicles/frame is a count, not a correctness score, so the default stays until real labels exist.

Conclusion & Future Work

what it proves, what is next

A single camera and a laptop GPU are enough to measure a road in real engineering units and drive real signal hardware from that measurement, at **27.1 fps end to end** with **±5% speed accuracy** once calibrated.

Almost every design decision here exists because something went wrong first, got measured, and got fixed — an empty road reporting HEAVY, a slow computer reporting slow cars, 29% of traffic reported as police. That trail, and the 833 checks that hold it down, is the real result.

Next:

- **A full four-way junction**. One controller drives one approach; four plus a coordinator is the extension, and request_phase() already exists for it.
- **A few hundred labelled frames**, to settle the 11s-vs-11m question with a real mAP instead of a vehicle count.
- **Retrain the livery model per camera**. On the camera it was built for, the trained classifier beats CLIP (71% vs 41–53% recognition) at a seventh of the cost — CLIP stays the default only because it travels.